DAVINE TECHNOLOGIES

INNOVATED. BUILD. ELEVATE.

PROJECT REPORT

NEXVION

Project Repository: https://github.com/CloudWithC/Nexvion

Project Title	Nexvion – End-to-End DevOps Implementation
Internship Domain	DevOps
Submitted By	M.V Cibistha
Submission Date	30.10.2026

TABLE OF CONTENTS

No.	Section	Page No.
1	Project Overview	3
2	Problem Statement	4
3	Project Objectives	5
4	Project Scope	6
5	Technologies & Tools Used	8
6	System / Project Architecture	10
7	Project Workflow	12
8	Project Implementation	14
9	Key Features / Functionalities	21
10	Testing & Validation	22
11	Results & Outcomes	24
12	Project Screenshots & Evidence	25
13	Challenges & Solutions	28
14	Learning Outcomes	30
15	Conclusion	32
16	Future Enhancements	33
17	References	34

1. Project Overview

1.1 Project Description

Nexvion is a web-based e-commerce demonstration application developed and used as the foundation for an end-to-end DevOps implementation project during the internship at Davine Technologies.

The project focuses on applying DevOps practices to an application throughout its development and deployment lifecycle. Instead of concentrating only on application development, the project demonstrates how an application can be containerized, integrated with automated pipelines, configured through infrastructure automation, deployed to a local Kubernetes environment, monitored, and scanned for security issues.

The Nexvion application contains product listings, search and filtering functionality, shopping cart operations, user login and registration interfaces, and a separate payment demonstration page.

The DevOps implementation was divided into six major phases:

1. Docker and CI/CD implementation.
2. Infrastructure as Code using Terraform.
3. Configuration management using Ansible.
4. Container orchestration using Kubernetes.
5. Monitoring and observability.
6. Security scanning and validation.

The implementation uses a combination of local infrastructure, cloud configuration, automation tools, and GitHub-based workflows.

The main objective is to demonstrate a practical DevOps lifecycle using a real application rather than studying individual tools independently.

1.2 Use Case

The project demonstrates a workflow in which a developer makes changes to the Nexvion application and pushes them to the GitHub repository.

The automated CI/CD pipeline performs the required checks, builds the Docker image, and publishes the image to GitHub Container Registry (GHCR).

A self-hosted GitHub Actions runner is used for deployment. The application is deployed using separate candidate and production environments. Health checks are performed to verify application availability, and rollback functionality is used to recover from an unsuccessful deployment.

Terraform is used to define AWS infrastructure configuration, while Ansible automates server configuration. Kubernetes demonstrates container orchestration, including deployments, services, configuration, and rollback.

Prometheus, Grafana, Node Exporter, Blackbox Exporter, and Alertmanager are used to monitor application and system health. Trivy is used to scan the project filesystem and Docker image for security-related issues.

> ***Overall use case:*** *Automate the application delivery process while maintaining deployment reliability, infrastructure consistency, visibility, and security.*

2. Problem Statement

2.1 Existing Problem

In a traditional application deployment process, developers may need to build, configure, test, and deploy applications manually.

This creates several operational problems:

1. **Manual deployment:** Repeating deployment steps manually increases effort and the possibility of human error.
2. **Environment inconsistency:** Applications may behave differently because development and deployment environments are configured differently.
3. **Difficult rollback:** Recovering from an unsuccessful deployment can require manual intervention.
4. **Limited infrastructure automation:** Manually creating and configuring infrastructure makes it difficult to reproduce environments consistently.
5. **Lack of centralized monitoring:** Without proper monitoring, application failures and resource-related problems may not be identified quickly.
6. **Security risks:** Applications and container images may contain vulnerable dependencies or accidentally exposed secrets.
7. **Limited deployment visibility:** Without automated pipeline logs and monitoring dashboards, understanding the status of a deployment becomes difficult.

These problems affect application reliability, deployment speed, maintenance, and security.

2.2 Proposed Approach

The proposed solution is to implement a DevOps workflow for the Nexvion application by combining automation, containerization, infrastructure management, orchestration, monitoring, and security scanning.

The solution includes:

- Docker to package the application into a consistent container.
- Docker Compose to manage the application container configuration.
- GitHub Actions to automate CI/CD.
- GitHub Container Registry to store Docker images.
- A self-hosted runner to execute deployment operations.
- Terraform to define AWS infrastructure resources.
- Ansible to automate server configuration.
- Kubernetes with Kind to demonstrate container orchestration locally.
- Prometheus and Grafana to monitor infrastructure and application health.
- Trivy to perform filesystem, secret, and container image scanning.

This approach reduces repetitive manual work and provides a structured method for building, testing, deploying, monitoring, and securing the application.

3. Project Objectives

3.1 Objectives

The main objectives of the Nexvion DevOps project are:

1. **Application Containerization:** Package the Nexvion application using Docker.
2. **Continuous Integration:** Automate application checks and build processes using GitHub Actions.
3. **Continuous Deployment:** Automate application deployment using a self-hosted runner.
4. **Container Image Management:** Publish and manage Docker images through GitHub Container Registry.
5. **Deployment Reliability:** Implement application health checks and rollback procedures.
6. **Infrastructure as Code:** Define infrastructure configuration using Terraform.
7. **Configuration Management:** Automate server configuration using Ansible.
8. **Container Orchestration:** Deploy and manage application containers using Kubernetes.
9. **Monitoring:** Track CPU, memory, disk, and application availability.
10. **Alerting:** Configure alerts for important application and system conditions.
11. **Security:** Scan project files and Docker images using Trivy.
12. **Practical DevOps Experience:** Understand how different DevOps tools work together in an application delivery lifecycle.



4. Project Scope

4.1 Included Scope

The project includes the following implementation activities:

Application and Containerization

- Use the Nexvion frontend application.
- Create a Dockerfile.
- Build and run Docker images.
- Configure Docker Compose.
- Verify application pages through HTTP requests.

CI/CD

- Create GitHub Actions workflows.
- Automate application checks and Docker image building.
- Publish images to GHCR.
- Use a self-hosted runner for deployment.
- Configure candidate and production deployment ports.
- Implement health checks and rollback.

Infrastructure Automation

- Define AWS security group configuration using Terraform.
- Use variables and outputs.
- Automate Ubuntu server configuration using Ansible.
- Install and configure Nginx.

Kubernetes

- Create a local Kubernetes cluster using Kind.
- Deploy the application in a dedicated namespace.
- Configure Deployment, ConfigMap, Secret placeholder, and NodePort Service.
- Test replicas, application access, rolling restart, and rollback.

Monitoring and Observability

- Configure Prometheus and Node Exporter.
- Configure Grafana dashboards.
- Configure Blackbox Exporter for website availability checks.
- Configure Alertmanager.
- Test website failure and recovery alerts.

Security

- Configure Trivy filesystem and secret scanning.
- Configure Docker image scanning.
- Enforce security scan results through GitHub Actions.

4.2 Limitations

The project was implemented as an internship demonstration and has the following limitations:

1. The Nexvion application is a frontend demonstration and does not use a production backend.
2. Login and registration functionality are demonstration features using browser local storage.
3. The payment page does not process real financial transactions.
4. Terraform infrastructure configuration was developed for AWS, but a complete production EC2 deployment was not performed.
5. Kubernetes implementation was performed locally using Kind rather than a managed production Kubernetes service.
6. The Kubernetes Secret contains demonstration placeholder information and is not intended for production use.

5. Technologies & Tools Used

5.1 Technology Stack

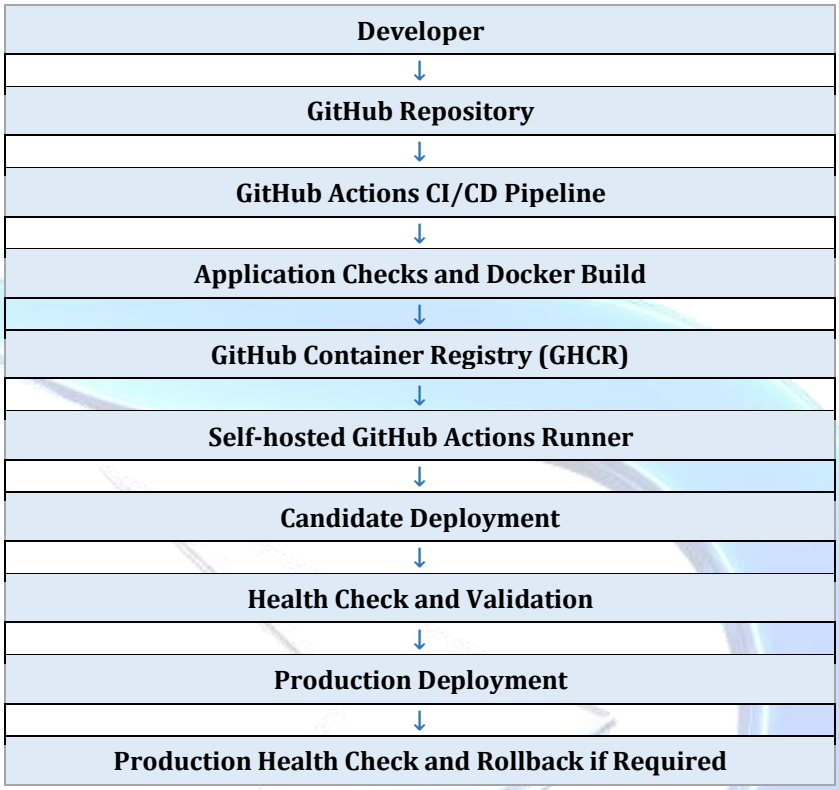
Technology	Purpose
HTML	Application page structure
CSS	Application styling
JavaScript	Frontend functionality
Docker	Application containerization
Docker Compose	Container configuration
Git	Version control
GitHub	Source code hosting
GitHub Actions	CI/CD automation
GitHub Container Registry	Docker image storage
Self-hosted Runner	Automated deployment execution
Linux / Ubuntu	Server environment
Terraform	Infrastructure as Code
AWS	Cloud infrastructure configuration
Ansible	Configuration management
Nginx	Web server and reverse proxy
Kubernetes	Container orchestration
Kind	Local Kubernetes cluster
Prometheus	Metrics collection
Grafana	Monitoring dashboards
Node Exporter	System metrics
Blackbox Exporter	Website availability monitoring
Alertmanager	Alert management
Trivy	Security scanning

6. System / Project Architecture

6.1 Architecture Diagram

The architecture represents the complete workflow from source code changes to deployment, monitoring, and security validation.

Architecture Flow:



The supporting DevOps components operate alongside this deployment workflow:

- Terraform defines AWS infrastructure configuration.
- Ansible configures the Ubuntu server.
- Kubernetes manages local container orchestration.
- Prometheus collects monitoring metrics.
- Grafana displays monitoring dashboards.
- Blackbox Exporter checks website availability.
- Alertmanager manages alerts.
- Trivy scans files and container images.

7. Project Workflow

7.1 End-to-End Workflow

The project was completed through six major implementation phases.

Phase 1: Docker and CI/CD

1. Prepared the Nexvion application.
2. Created a Dockerfile.
3. Built the Docker image.
4. Tested the application using Docker.
5. Configured Docker Compose.
6. Created GitHub Actions CI/CD workflow.
7. Configured GHCR image publishing.
8. Connected a self-hosted runner.
9. Deployed the candidate application.
10. Performed health checks.
11. Deployed the production application.
12. Tested rollback functionality.

Phase 2: Terraform

1. Prepared Terraform configuration.
2. Defined the AWS provider and region.
3. Created security group configuration.
4. Used variables to manage configuration values.
5. Defined outputs to display relevant resource information.
6. Maintained local Terraform state and plan files outside Git tracking.

Phase 3: Ansible

1. Prepared the Ansible inventory.
2. Connected to the Ubuntu environment.
3. Created a playbook for server configuration.
4. Automated package installation.
5. Installed and configured Nginx.
6. Used a Jinja2 template.
7. Executed the playbook.
8. Verified successful execution and idempotency.

Phase 4: Kubernetes

1. Created a Kind cluster.
2. Created the `nexvion` namespace.
3. Created the application Deployment.
4. Configured two replicas.
5. Created a ConfigMap.
6. Added a demonstration Secret placeholder.
7. Created a NodePort Service.
8. Verified application access using port forwarding.
9. Tested rolling restart.
10. Simulated a bad image deployment.
11. Performed rollback and verified recovery.

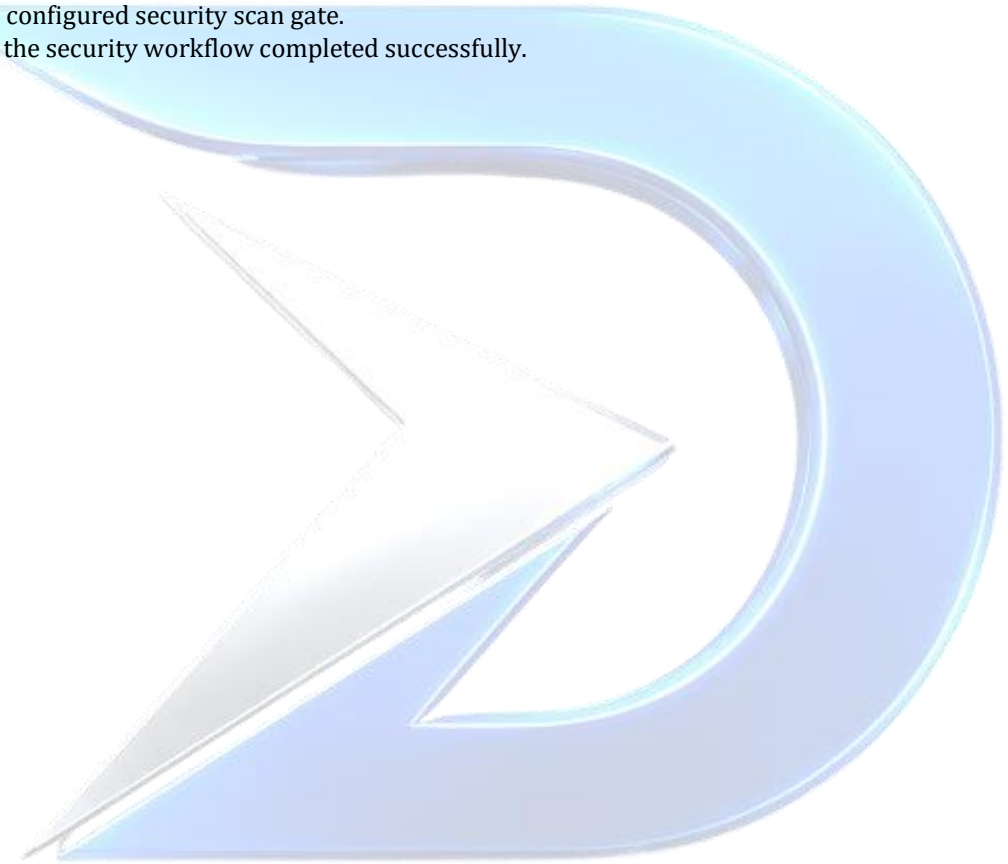
Phase 5: Monitoring and Observability

1. Configured Prometheus.
2. Configured Node Exporter.
3. Configured Grafana.
4. Added Blackbox Exporter.

5. Configured Alertmanager.
6. Verified monitoring targets.
7. Created a Grafana dashboard.
8. Added CPU, memory, disk, website availability, and HTTP status panels.
9. Created alert rules.
10. Simulated website failure.
11. Verified the alert.
12. Restarted the application and verified recovery.

Phase 6: Security

1. Created a dedicated GitHub Actions security workflow.
2. Configured Trivy filesystem scanning.
3. Enabled vulnerability and secret scanning.
4. Configured severity filtering.
5. Added Docker image scanning.
6. Enforced the configured security scan gate.
7. Verified that the security workflow completed successfully.



8. Project Implementation

8.1 Environment & Initial Setup

The project was maintained in a GitHub repository named `Nexvion`.

Repository: https://github.com/CloudWithC/Nexvion

Development environment:

- Operating System: Ubuntu Linux
- Version Control: Git
- Container Platform: Docker
- CI/CD Platform: GitHub Actions
- Deployment Environment: Ubuntu server with self-hosted runner
- Infrastructure Tool: Terraform
- Configuration Tool: Ansible
- Orchestration Tool: Kubernetes with Kind
- Monitoring Tools: Prometheus and Grafana
- Security Tool: Trivy

The project source code was maintained in the `main` branch.

The local working directory used for the project was `~/nexvion-devops`.

The repository contained application files, Docker configuration, GitHub Actions workflows, Terraform configuration, Ansible playbooks, Kubernetes manifests, and project documentation.

Phase 1 – Docker and CI/CD Implementation

The first phase focused on containerizing the Nexvion application and automating its deployment.

A Dockerfile was created to package the frontend application. Docker Compose was used to define the container configuration.

The application was tested through its main pages:

- `index.html`
- `products.html`
- `payment.html`

A GitHub Actions workflow was created at `.github/workflows/ci.yml`. The workflow automated application checks, Docker image building, and publishing to GitHub Container Registry.

A self-hosted GitHub Actions runner was configured on the Ubuntu server. This allowed deployment operations to execute on the project server.

The deployment workflow used two environments:

Environment	Port
Candidate	8089
Production	8088

The candidate application was deployed first. Health checks were performed before proceeding with production deployment.

The workflow also included rollback functionality so that an unsuccessful deployment could be recovered.

Important implementation evidence:

- CI/CD workflow run #18 completed successfully before the README documentation update.
- The deployment workflow used the self-hosted runner.
- Application health checks were performed for the main, products, and payment pages.
- The latest pipeline run after the final README update should be verified separately before claiming that particular run passed.

Phase 2 – Terraform Infrastructure as Code

The second phase focused on defining infrastructure using Terraform.

Terraform configuration was prepared for an AWS security group in the `eu-north-1` region.

The configuration included:

- AWS provider configuration.
- Security group resource.
- Input variables.
- Output values.

The configured security group ID was `sg-0ca97a39595ebc411`.

Terraform variables were used to make the configuration reusable, while outputs were used to display relevant resource information.

Terraform state and plan files were maintained locally and excluded from Git tracking.

The project demonstrated the Infrastructure as Code approach, in which infrastructure configuration is represented through code rather than being managed entirely through manual console operations.

> ***Scope note:*** *The project did not provision a complete EC2-based production environment. The Terraform work focused on AWS security group configuration and infrastructure code.*

Phase 3 – Ansible Configuration Management

The third phase focused on automating server configuration using Ansible.

Ansible inventory and playbook files were prepared to manage the Ubuntu environment.

The playbook automated tasks such as:

- Installing required packages.
- Installing Nginx.
- Configuring the web server.
- Using a Jinja2 template.
- Creating and managing application-related configuration.

The playbook was executed against the configured environment.

The final idempotency verification produced `ok=9 changed=0 failed=0 unreachable=0`. This showed that the playbook completed without failures and that a repeated execution did not require further configuration changes.

The Ansible workflow was also integrated into GitHub Actions. The workflow passed after pinning Python 3.12 to resolve the Python-version compatibility issue.

Phase 4 – Kubernetes Container Orchestration

The fourth phase focused on container orchestration using Kubernetes.

A local Kubernetes cluster was created using Kind. The cluster name was `cib-cluster`. A dedicated namespace named `nexvion` was created to organize the application resources.

The Kubernetes implementation included:

- Deployment.
- Two application replicas.
- ConfigMap.
- Demonstration Secret placeholder.

- NodePort Service.

The application Deployment used the image `nexvion-web:v1`. The NodePort Service exposed the application through port `30088`.

The application was tested using port forwarding through `localhost:18088`. The main application pages returned successful HTTP responses.

A rolling restart was performed to verify the Deployment's ability to restart application pods.

A failure scenario was also tested by simulating an invalid image version, `nexvion-web:bad`. The failed deployment was rolled back to the previous working version. After rollback, the application returned to a healthy state with two running replicas.

This phase demonstrated deployment management, replica management, service exposure, rolling restart, and recovery through rollback.

Phase 5 – Monitoring and Observability

The fifth phase focused on monitoring the application and server environment.

The following monitoring components were configured:

Component	Port	Purpose
Prometheus	9090	Metrics collection
Node Exporter	9100	Linux system metrics
Grafana	3000	Monitoring dashboards
Blackbox Exporter	9115	Website availability checks
Alertmanager	9093	Alert management

Prometheus was used to collect metrics from the configured monitoring targets. Node Exporter provided system metrics such as CPU, memory, and disk usage. Blackbox Exporter was used to check website availability and HTTP responses.

A Grafana dashboard named `Nexvion Monitoring Dashboard` was created with five monitoring panels:

1. CPU Usage.
2. Memory Usage.
3. Disk Usage.
4. Website Availability.
5. HTTP Status.

Four Prometheus alert rules were configured:

- `NexvionWebsiteDown`
- `HighCPUUsage`
- `HighMemoryUsage`
- `LowDiskSpace`

To validate website failure detection, the candidate application container was stopped. The `NexvionWebsiteDown` alert was observed firing for the candidate endpoint on port `8089`. The application was then restarted, and HTTP status 200 was verified after recovery.

This demonstrated that the monitoring setup could detect application unavailability and show recovery after the service was restored.

Alertmanager was configured for local alert management. External email and SMS notification integrations were not implemented.

Phase 6 – Security Scanning

The sixth phase focused on integrating security scanning into the CI/CD process.

A dedicated workflow was created at `.github/workflows/security.yml`. The workflow was named `Nexvion Security Scan`.

Trivy was configured to perform filesystem scanning and Docker image scanning. The filesystem scan included vulnerability and secret scanning:

- Scan type: `fs`
- Scanners: `vuln,secret`
- Severity levels: `HIGH,CRITICAL`
- Ignore unfixed vulnerabilities: `true`
- Exit code: `1`

The configured security gate was intended to fail the workflow when findings meeting the selected criteria were detected.

Docker image scanning was also configured to inspect operating system and application library vulnerabilities.

The security workflow contained two important scan jobs:

1. Filesystem and secret scanning.
2. Docker image scanning.

Both jobs were verified as successful in the dedicated GitHub Actions security workflow.

The project also contained a Docker image scan in the CI workflow. This provided an additional image scanning step.

The security implementation demonstrated how automated scanning can be included in the software delivery pipeline rather than being performed only as a separate manual activity.

> *A successful workflow indicates that the scans passed under the configured rules. It should not be interpreted as a guarantee that the application contains no vulnerabilities.*

8.2 Module / Feature Implementation

The implementation was divided into six modules.

Module	Implementation
Module 1	Docker containerization and automated CI/CD deployment
Module 2	AWS infrastructure configuration using Terraform
Module 3	Ubuntu server configuration using Ansible
Module 4	Kubernetes deployment and orchestration using Kind
Module 5	Monitoring and alerting using Prometheus and Grafana
Module 6	Filesystem, secret, and Docker image scanning using Trivy

Each module addressed a different part of the DevOps lifecycle. Together, they formed a complete demonstration workflow.

8.3 Deployment / Execution

The application deployment process followed these steps:

1. Application source code was maintained in the GitHub repository.
2. Code changes were pushed to the `main` branch.
3. GitHub Actions started the configured CI/CD workflow.
4. Application checks and Docker image build operations were executed.

5. The Docker image was published to GHCR.
6. The self-hosted runner executed deployment operations.
7. The candidate application was deployed on port `8089`.
8. Health checks were performed.
9. The production application was deployed on port `8088`.
10. Production application endpoints were verified.
11. Rollback functionality was available to recover from unsuccessful deployments.

The Kubernetes environment was tested separately using the local Kind cluster.

Monitoring and security workflows were also executed to validate application health and scan the project.



9. Key Features / Functionalities

9.1 Feature List

1. Containerized Application
The Nexvion application was packaged into a Docker image for consistent execution.

2. Automated CI/CD Pipeline
GitHub Actions automated application checks, image building, and publishing.

3. Container Image Publishing
Docker images were published to GitHub Container Registry.

4. Candidate and Production Deployment
Separate deployment environments were maintained using ports 8089 and 8088.

5. Health Checks
HTTP checks were used to verify that application pages were accessible.

6. Rollback Support
Rollback procedures were used to recover from unsuccessful application deployments.

7. Infrastructure as Code
Terraform configuration was used to define AWS security group infrastructure.

8. Automated Configuration Management
Ansible automated Ubuntu server setup and Nginx configuration.

9. Kubernetes Deployment
The application was deployed in a local Kubernetes cluster with two replicas.

10. Kubernetes Service Exposure
A NodePort service was used to expose the application.

11. Kubernetes Recovery
Rolling restart and rollback operations were tested.

12. Monitoring Dashboard
Grafana displayed CPU, memory, disk, website availability, and HTTP status.

13. Alerting
Prometheus alert rules were configured for website availability and system resource conditions.

14. Security Scanning
Trivy scanned the project filesystem, secrets, and Docker image.

15. Version Control and Documentation
GitHub maintained source code, workflow files, and project documentation.

10. Testing & Validation

10.1 Testing Approach

Testing was performed at different stages of the project to verify the functionality and reliability of each DevOps component.

The testing approach included:

- Application page verification.
- Docker container testing.
- CI/CD workflow validation.
- Candidate and production health checks.
- Terraform configuration verification.
- Ansible execution and idempotency testing.
- Kubernetes deployment and rollback testing.
- Monitoring target verification.
- Alert failure and recovery testing.
- Trivy security workflow validation.

Testing was performed using command-line tools, GitHub Actions results, monitoring dashboards, and application HTTP responses.

10.2 Test Cases

Test ID	Test Case	Expected Result	Observed Result
TC01	Build Docker image	Image should build successfully	Verified
TC02	Run Nexvion container	Application should start	Verified
TC03	Access main page	HTTP success response	Verified
TC04	Access products page	HTTP success response	Verified
TC05	Access payment page	HTTP success response	Verified
TC06	Execute CI/CD workflow	Workflow should complete successfully	Successful run verified
TC07	Publish Docker image	Image should be available in GHCR	Verified
TC08	Candidate deployment	Application should run on port 8089	Verified
TC09	Production deployment	Application should run on port 8088	Verified
TC10	Application health check	Healthy pages should return HTTP 200	Verified
TC11	Terraform configuration	Infrastructure configuration should be prepared	Verified as project configuration
TC12	Ansible execution	Playbook should complete without failures	Verified
TC13	Ansible idempotency	Repeated run should not make unnecessary changes	`ok=9 changed=0 failed=0`
TC14	Kubernetes deployment	Application pods should become available	Verified
TC15	Kubernetes replicas	Two replicas should run	Verified
TC16	Kubernetes service	Application should be accessible through service/port forwarding	Verified
TC17	Kubernetes rolling restart	Deployment should restart successfully	Verified
TC18	Kubernetes failure simulation	Invalid image should cause deployment failure	Verified

Test ID	Test Case	Expected Result	Observed Result
TC19	Kubernetes rollback	Previous working deployment should be restored	Verified
TC20	Prometheus monitoring	Configured targets should be UP	Verified
TC21	Grafana dashboard	Monitoring panels should display metrics	Verified
TC22	Website failure alert	Website-down alert should fire	Verified
TC23	Website recovery	Website should become accessible after restart	Verified
TC24	Trivy filesystem scan	Configured scan should complete successfully	Verified
TC25	Trivy Docker image scan	Configured image scan should complete successfully	Verified

> *Note: "Verified" refers to the practical results observed during implementation. The report screenshots should be used as supporting evidence.*

10.3 Validation

The project was validated through successful execution of the configured workflows and practical tests.

The Docker and CI/CD implementation was validated through image building, GHCR publishing, application deployment, and HTTP health checks.

Terraform configuration was prepared for AWS security group management.

Ansible execution completed successfully with zero failed and unreachable tasks. The repeated run reported zero changes, demonstrating idempotency.

Kubernetes validation included successful deployment, two running replicas, application access, rolling restart, failure simulation, and rollback.

Monitoring validation confirmed that configured Prometheus targets were UP and that Grafana displayed the required monitoring panels. The website-down alert was observed during the failure test, and application recovery was verified after restarting the container.

The Trivy filesystem/secret and Docker image scanning jobs completed successfully under the configured security rules.

11. Results & Outcomes

11.1 Final Results

The Nexvion project successfully demonstrated the integration of multiple DevOps tools into a structured application delivery workflow.

The major results were:

1. The Nexvion frontend application was containerized using Docker.
2. Docker Compose was configured for container management.
3. GitHub Actions automated the CI/CD workflow.
4. Docker images were published to GHCR.
5. A self-hosted runner was used for deployment.
6. Candidate and production deployment environments were configured.
7. Application health checks and rollback procedures were implemented.
8. Terraform infrastructure configuration was prepared for AWS.
9. Ansible automated Ubuntu server configuration.
10. Kubernetes deployment and rollback were tested locally using Kind.
11. Prometheus and Grafana provided monitoring and visualization.
12. Website failure and recovery alerts were tested.
13. Trivy scanning was integrated into GitHub Actions.
14. Project implementation details were documented in the GitHub repository.

The final GitHub repository was updated with the project documentation and implementation files.

11.2 Outcome Evaluation

The project demonstrated how individual DevOps tools can work together to improve the application delivery process.

Area	Outcome
Containerization	Application packaged using Docker
CI/CD	Automated workflow configured
Image Management	GHCR publishing implemented
Deployment	Candidate and production environments configured
Recovery	Health checks and rollback tested
Infrastructure	Terraform AWS configuration prepared
Configuration	Ansible automation completed
Orchestration	Local Kubernetes deployment validated
Monitoring	Prometheus and Grafana configured
Alerting	Failure and recovery scenario tested
Security	Trivy scans integrated into CI/CD
Documentation	GitHub README updated

The project provided practical experience in automation, deployment management, infrastructure configuration, monitoring, troubleshooting, and security scanning.

12. Project Screenshots & Evidence

12.1 Implementation Evidence

The following screenshots should be inserted to demonstrate the implementation of each phase

Phase 1 – Docker and CI/CD

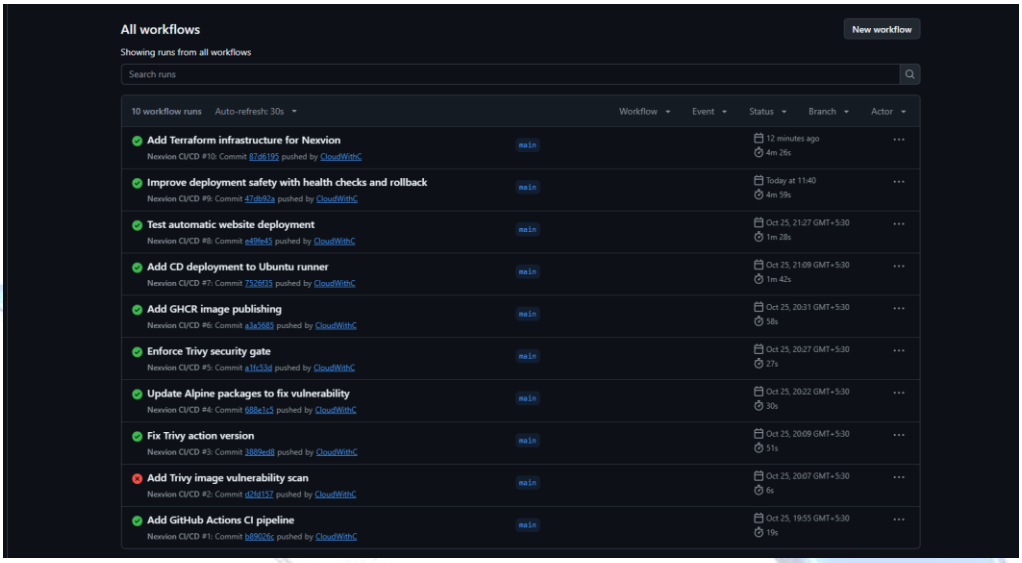
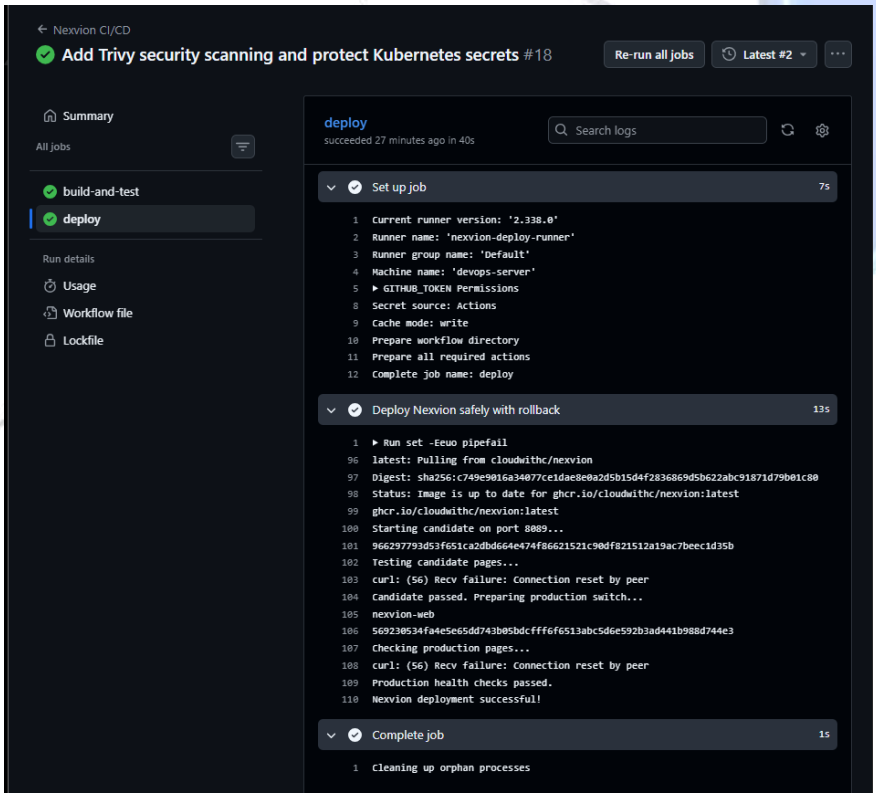


Figure 12.1: *Successful GitHub Actions CI/CD workflow.*



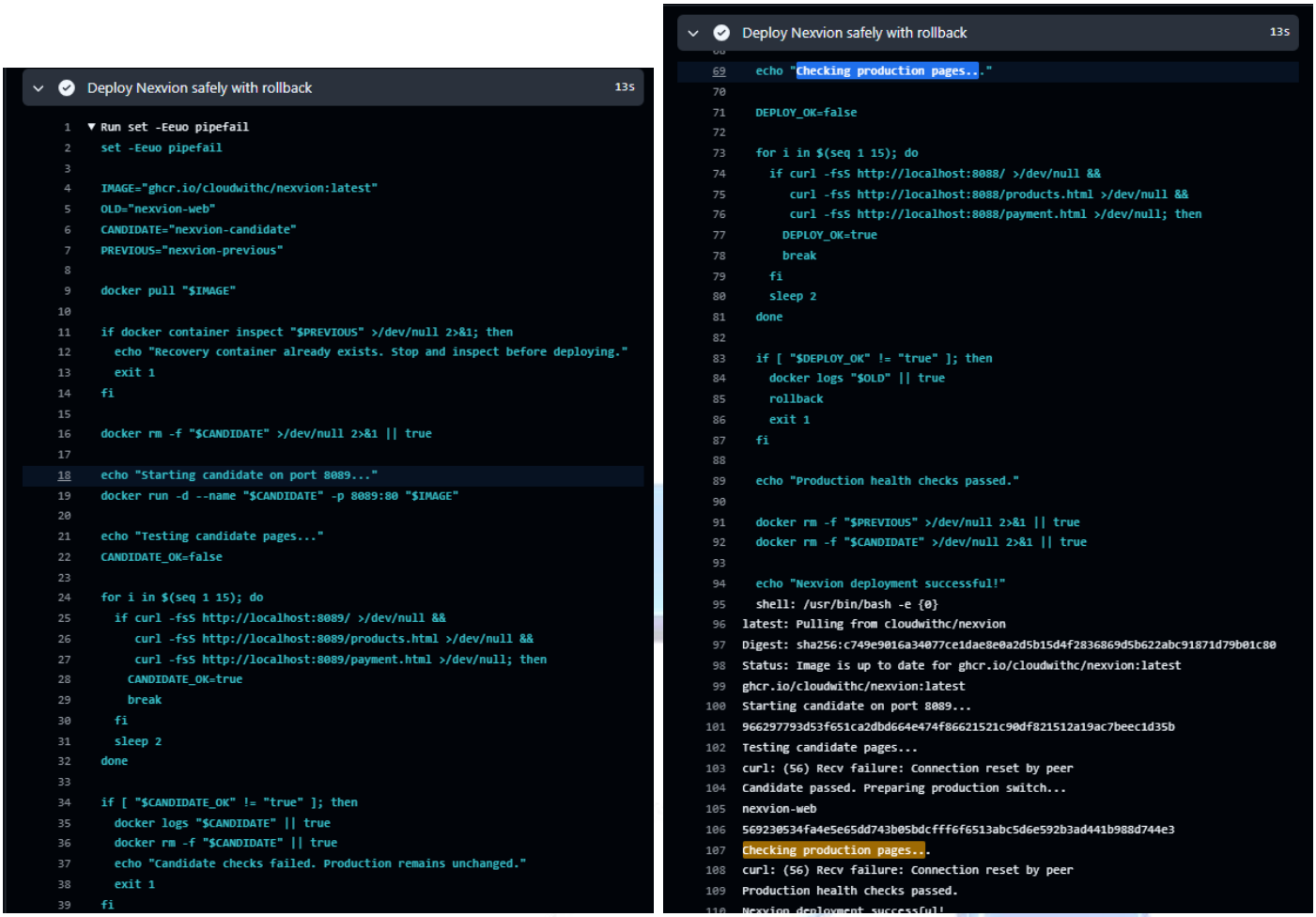


Figure 12.2: *Candidate and production deployment health checks.*

Phase 2 – Terraform

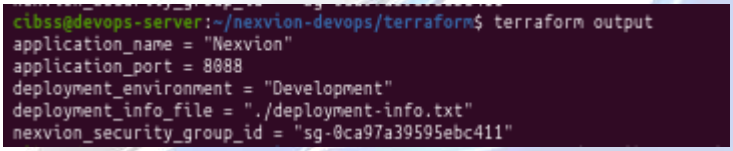
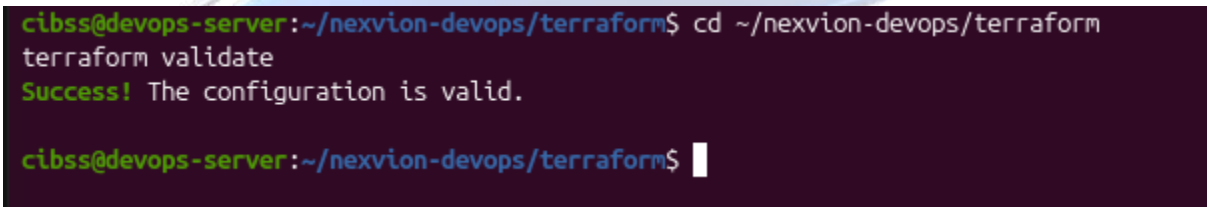


Figure 12.3: *Terraform variables and outputs.*



```
cibss@devops-server:~/nexvion-devops/terraform$ terraform apply
local_file.deployment_info: Refreshing state... [id=0b90f5ccfba3118993a0ebd6c4aef0c47e2e5483]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_security_group.nexvion_web will be created
  + resource "aws_security_group" "nexvion_web" {
      + arn                    = (known after apply)
      + description            = "Security group for Nexvion web application"
      + egress                 = [
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + description      = "Allow outbound traffic"
              + from_port        = 0
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "-1"
              + security_groups  = []
              + self             = false
              + to_port          = 0
            },
        ]
      + id                     = (known after apply)
      + ingress                = [
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + description      = "Allow HTTP traffic"
              + from_port        = 80
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "tcp"
              + security_groups  = []
              + self             = false
              + to_port          = 80
            },
        ]
      + name                   = "nexvion-web-sg"
      + name_prefix            = (known after apply)
      + owner_id               = (known after apply)
      + region                 = "eu-north-1"
      + revoke_rules_on_delete = false
      + tags                   = {
          + "Environment" = "Development"
          + "ManagedBy"   = "Terraform"
          + "Name"        = "nexvion-web-sg"
          + "Project"     = "Nexvion"
        }
      + tags_all               = {
          + "Environment" = "Development"
          + "ManagedBy"   = "Terraform"
          + "Name"        = "nexvion-web-sg"
          + "Project"     = "Nexvion"
        }
      + vpc_id                 = "vpc-0b7f2298f6f7c57f9"
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + nexvion_security_group_id = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_security_group.nexvion_web: Creating...
aws_security_group.nexvion_web: Creation complete after 5s [id=sg-0ca97a39595ebc411]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

application_name = "Nexvion"
application_port = 8080
deployment_environment = "Development"
deployment_info_file = "./deployment-info.txt"
```

```
Success! The configuration is valid.

local_file.deployment_info: Refreshing state... [id=0b90f5ccfba3118993a0ebd6c4aef0c47e2e5483]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_security_group.nexvion_web will be created
  + resource "aws_security_group" "nexvion_web" {
      + arn                    = (known after apply)
      + description            = "Security group for Nexvion web application"
      + egress                 = [
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + description      = "Allow outbound traffic"
              + from_port        = 0
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "-1"
              + security_groups  = []
              + self             = false
              + to_port          = 0
            },
        ]
      + id                     = (known after apply)
      + ingress                = [
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + description      = "Allow HTTP traffic"
              + from_port        = 80
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "tcp"
              + security_groups  = []
              + self             = false
              + to_port          = 80
            },
        ]
      + name                   = "nexvion-web-sg"
      + name_prefix            = (known after apply)
      + owner_id               = (known after apply)
      + region                 = "eu-north-1"
      + revoke_rules_on_delete = false
      + tags                   = {
          + "Environment" = "Development"
          + "ManagedBy"   = "Terraform"
          + "Name"        = "nexvion-web-sg"
          + "Project"     = "Nexvion"
        }
      + tags_all               = {
          + "Environment" = "Development"
          + "ManagedBy"   = "Terraform"
          + "Name"        = "nexvion-web-sg"
          + "Project"     = "Nexvion"
        }
      + vpc_id                 = "vpc-0b7f2298f6f7c57f9"
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + nexvion_security_group_id = (known after apply)
```

```
cibss@devops-server:~/nexvion-devops/terraform$ aws ec2 describe-security-groups \
  --region eu-north-1 \
  --group-ids sg-0ca97a39595ebc411 \
  --query 'SecurityGroups[0].{Name:GroupName,VPC:VpcId,Ingress:IpPermissions,Egress:IpPermissionsEgress,Tags:Tags}' \
  --output json
{
    "Name": "nexvion-web-sg",
    "VPC": "vpc-0b7f2298f6f7c57f9",
    "Ingress": [
        {
            "IpProtocol": "tcp",
            "FromPort": 80,
            "ToPort": 80,
            "UserIdGroupPairs": [],
            "IpRanges": [
                {
                    "Description": "Allow HTTP traffic",
                    "CidrIp": "0.0.0.0/0"
                }
            ],
            "Ipv6Ranges": [],
            "PrefixListIds": []
        }
    ],
    "Egress": [
        {
            "IpProtocol": "-1",
            "UserIdGroupPairs": [],
            "IpRanges": [
                {
                    "Description": "Allow outbound traffic",
                    "CidrIp": "0.0.0.0/0"
                }
            ],
            "Ipv6Ranges": [],
            "PrefixListIds": []
        }
    ],
    "Tags": [
        {
            "Key": "Name",
            "Value": "nexvion-web-sg"
        },
        {
            "Key": "Environment",
            "Value": "Development"
        },
        {
            "Key": "ManagedBy",
            "Value": "Terraform"
        },
        {
            "Key": "Project",
            "Value": "Nexvion"
        }
    ]
}
```

Figure 12.4: *Terraform validation or plan/resource evidence.*

Phase 3 – Ansible

```
cibss@devops-server:~/nexvion-devops/ansible$ cd ~/nexvion-devops/ansible
ansible-playbook playbooks/setup-nexvion.yml --syntax-check

playbook: playbooks/setup-nexvion.yml
cibss@devops-server:~/nexvion-devops/ansible$
```

Figure 12.5: *Ansible inventory and playbook.*

```
cibss@devops-server:~/nexvion-devops/ansible$ cd ~/nexvion-devops/ansible

sudo ansible-playbook playbooks/setup-nexvion.yml \
  -e ansible_become=false

PLAY [Configure Nexvion web server] ********************************************************************************

TASK [Gathering Facts] *********************************************************************************************
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of another Python interpreter could cause a dif
ferent interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [Update package cache] ****************************************************************************************
ok: [localhost]

TASK [Install Nginx] ***********************************************************************************************
ok: [localhost]

TASK [Create Nexvion web directory] ********************************************************************************
changed: [localhost]

TASK [Copy Nexvion website files] **********************************************************************************
changed: [localhost]

TASK [Configure Nexvion Nginx site] ********************************************************************************
changed: [localhost]

TASK [Enable Nexvion Nginx site] ***********************************************************************************
changed: [localhost]

TASK [Validate Nginx configuration] ********************************************************************************
ok: [localhost]

TASK [Ensure Nginx is running] *************************************************************************************
ok: [localhost]

TASK [Apply pending Nginx restart] *********************************************************************************

RUNNING HANDLER [Restart Nginx] ************************************************************************************
changed: [localhost]

PLAY RECAP *********************************************************************************************************
localhost                  : ok=10   changed=5    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

```
cibss@devops-server:~/nexvion-devops/ansible$ sudo systemctl status nginx --no-pager
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-10-30 16:50:29 IST; 51s ago
 Invocation: 121884da2312477dbc31fd437fae72fa
       Docs: man:nginx(8)
    Process: 15689 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 15691 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 15692 (nginx)
      Tasks: 3 (limit: 1856)
     Memory: 3.1M (peak: 3.6M)
        CPU: 61ms
     CGroup: /system.slice/nginx.service
             ├─15692 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             ├─15693 "nginx: worker process"
             └─15694 "nginx: worker process"

Oct 30 16:50:29 devops-server systemd[1]: Starting nginx.service - A high performance web server and a reverse …erver...
Oct 30 16:50:29 devops-server systemd[1]: Started nginx.service - A high performance web server and a reverse p… server.
Hint: Some lines were ellipsized, use -l to show in full.
```

```
cibss@devops-server:~/nexvion-devops/ansible$ curl -I http://localhost:8091
curl -I http://localhost:8091/products.html
curl -I http://localhost:8091/payment.html
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Fri, 30 Oct 2026 11:21:07 GMT
Content-Type: text/html
Content-Length: 11717
Last-Modified: Fri, 30 Oct 2026 11:20:21 GMT
Connection: keep-alive
ETag: "6ae47d75-2dc5"
Accept-Ranges: bytes

HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Fri, 30 Oct 2026 11:21:07 GMT
Content-Type: text/html
Content-Length: 9000
Last-Modified: Fri, 30 Oct 2026 11:20:23 GMT
Connection: keep-alive
ETag: "6ae47d77-2328"
Accept-Ranges: bytes

HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Fri, 30 Oct 2026 11:21:07 GMT
Content-Type: text/html
Content-Length: 8468
Last-Modified: Fri, 30 Oct 2026 11:20:24 GMT
Connection: keep-alive
ETag: "6ae47d78-2114"
Accept-Ranges: bytes
```

```
cibss@devops-server:~/nexvion-devops/ansible$ sudo ansible-playbook playbooks/setup-nexvion.yml -e ansible_become=false

PLAY [Configure Nexvion web server] ********************************************************************************

TASK [Gathering Facts] *********************************************************************************************
[WARNING]: Host 'localhost' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of another Python interpreter could cause a dif
ferent interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [Update package cache] ****************************************************************************************
ok: [localhost]

TASK [Install Nginx] ***********************************************************************************************
ok: [localhost]

TASK [Create Nexvion web directory] ********************************************************************************
ok: [localhost]

TASK [Copy Nexvion website files] **********************************************************************************
ok: [localhost]

TASK [Configure Nexvion Nginx site] ********************************************************************************
ok: [localhost]

TASK [Enable Nexvion Nginx site] ***********************************************************************************
ok: [localhost]

TASK [Validate Nginx configuration] ********************************************************************************
ok: [localhost]

TASK [Ensure Nginx is running] *************************************************************************************
ok: [localhost]

TASK [Apply pending Nginx restart] *********************************************************************************

PLAY RECAP *********************************************************************************************************
localhost                  : ok=9    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

cibss@devops-server:~/nexvion-devops/ansible$
```

Figure 12.7: *Ansible idempotency verification.*

Phase 4 – Kubernetes

```
^Ccibss@devops-server:~/nexvion-devops$ kubectl get deployments -n nexvion
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
nexvion   2/2     2            2           4m58s
cibss@devops-server:~/nexvion-devops$ kubectl get pods -n nexvion
NAME                       READY   STATUS    RESTARTS   AGE
nexvion-78ffc99c85-chcqb   1/1     Running   0          2m16s
nexvion-78ffc99c85-v8h29   1/1     Running   0          2m4s
cibss@devops-server:~/nexvion-devops$
```

Figure 12.8: *Kubernetes pods and Deployment.*

```
cibss@devops-server:~/nexvion-devops$ kubectl get services -n nexvion
NAME              TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
nexvion-service   NodePort   10.96.60.140   <none>        80:30088/TCP   6m37s
cibss@devops-server:~/nexvion-devops$
```

```
cibss@devops-server:~/nexvion-devops$ curl -I http://localhost:18088/
curl -I http://localhost:18088/products.html
curl -I http://localhost:18088/payment.html
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Fri, 30 Oct 2026 12:36:55 GMT
Content-Type: text/html
Content-Length: 11710
Last-Modified: Sun, 25 Oct 2026 13:57:32 GMT
Connection: keep-alive
ETag: "6ade0acc-2dbe"
Accept-Ranges: bytes

HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Fri, 30 Oct 2026 12:36:55 GMT
Content-Type: text/html
Content-Length: 9000
Last-Modified: Sun, 25 Oct 2026 13:57:32 GMT
Connection: keep-alive
ETag: "6ade0acc-2328"
Accept-Ranges: bytes

HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Fri, 30 Oct 2026 12:36:55 GMT
Content-Type: text/html
Content-Length: 8468
Last-Modified: Sun, 25 Oct 2026 13:57:32 GMT
Connection: keep-alive
ETag: "6ade0acc-2114"
Accept-Ranges: bytes

cibss@devops-server:~/nexvion-devops$
```

Figure 12.9: *Kubernetes Service and application access.*

```
cibss@devops-server:~/nexvion-devops$ kubectl set image deployment/nexvion nexvion=nexvion-web:bad -n nexvion
deployment.apps/nexvion image updated
cibss@devops-server:~/nexvion-devops$ kubectl get pods -n nexvion
kubectl rollout history deployment/nexvion -n nexvion
NAME                       READY   STATUS         RESTARTS   AGE
nexvion-78ffc99c85-chcqb   1/1     Running        0          5m16s
nexvion-78ffc99c85-v8h29   1/1     Running        0          5m4s
nexvion-7f4f99bcf5-gnw8p   0/1     ErrImagePull   0          9s
deployment.apps/nexvion
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
3         <none>

cibss@devops-server:~/nexvion-devops$ kubectl rollout undo deployment/nexvion -n nexvion
kubectl rollout status deployment/nexvion -n nexvion --timeout=120s
kubectl get deployments,pods -n nexvion
Warning: resource deployments/nexvion was previously managed with 'kubectl apply'. Rolling back will not update the kube
ctl.kubernetes.io/last-applied-configuration annotation, which may cause unexpected behavior on future 'kubectl apply' o
perations. Consider using 'kubectl apply' with your previous configuration file instead.
deployment.apps/nexvion rolled back
deployment "nexvion" successfully rolled out
NAME                       READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nexvion   2/2     2            2           8m34s

NAME                           READY   STATUS        RESTARTS   AGE
pod/nexvion-78ffc99c85-chcqb   1/1     Running       0          5m21s
pod/nexvion-78ffc99c85-v8h29   1/1     Running       0          5m9s
pod/nexvion-7f4f99bcf5-gnw8p   0/1     Terminating   0          14s
cibss@devops-server:~/nexvion-devops$ kubectl get deployments,pods -n nexvion
NAME                       READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nexvion   2/2     2            2           9m6s

NAME                           READY   STATUS    RESTARTS   AGE
pod/nexvion-78ffc99c85-chcqb   1/1     Running   0          5m53s
pod/nexvion-78ffc99c85-v8h29   1/1     Running   0          5m41s
cibss@devops-server:~/nexvion-devops$
```

Figure 12.10: *Kubernetes failure simulation and rollback.*

Phase 5 – Monitoring and Observability

```
cibss@devops-server:~/nexvion-devops$ echo "===== WEBSITE STATUS ====="
for port in 8088 8089 8091; do
  echo -n "Port $port: "
  curl -s -o /dev/null -w "%{http_code}\n" http://localhost:$port/
done

echo "===== PROMETHEUS TARGETS ====="
curl -s http://localhost:9090/api/v1/targets | python3 -c "
import json,sys
d=json.load(sys.stdin)
for t in d['data']['activeTargets']:
    print(t['labels'].get('job'), t['labels'].get('instance'), t['health'])
"
===== WEBSITE STATUS =====
Port 8088: 200
Port 8089: 200
Port 8091: 200
===== PROMETHEUS TARGETS =====
blackbox http://localhost:8088/ up
blackbox http://localhost:8089/ up
blackbox http://localhost:8091/ up
node localhost:9100 up
prometheus localhost:9090 up
```

Figure 12.11: *Prometheus targets showing UP status.*



Figure 12.12: *Grafana Nexvion Monitoring Dashboard.*

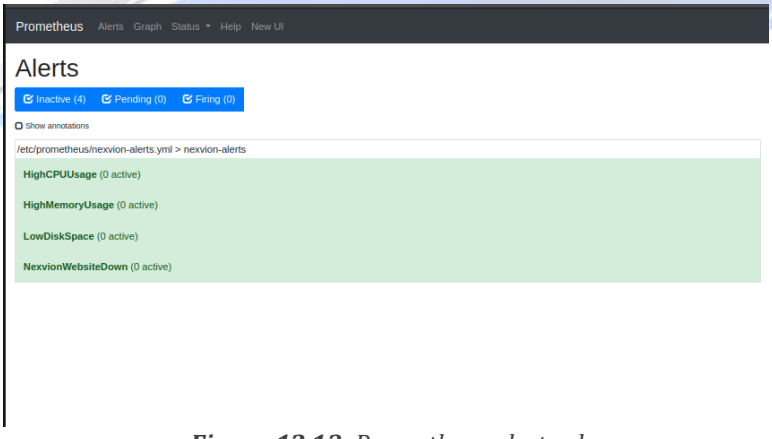


Figure 12.13: *Prometheus alert rules.*

```
cibss@devops-server:~/nexvion-devops$ echo "===== WEBSITE DOWN ALERT ====="
curl -s http://localhost:9090/api/v1/alerts | python3 -c "
import json,sys
d=json.load(sys.stdin)
for a in d['data']['alerts']:
    print(a['labels'].get('alertname'), a['state'], a['labels'].get('instance'))
"
===== WEBSITE DOWN ALERT =====
NexvionWebsiteDown firing http://localhost:8089/
```

Figure 12.14: *Website-down alert firing.*

```
cibss@devops-server:~/nexvion-devops$ docker start nexvion-candidate
nexvion-candidate
cibss@devops-server:~/nexvion-devops$ curl -s -o /dev/null -w "Candidate website: HTTP %{http_code}\n" http://localhost
8089/
Candidate website: HTTP 200
```

Figure 12.15: *Website recovery after restarting the application.*

Phase 6 – Security

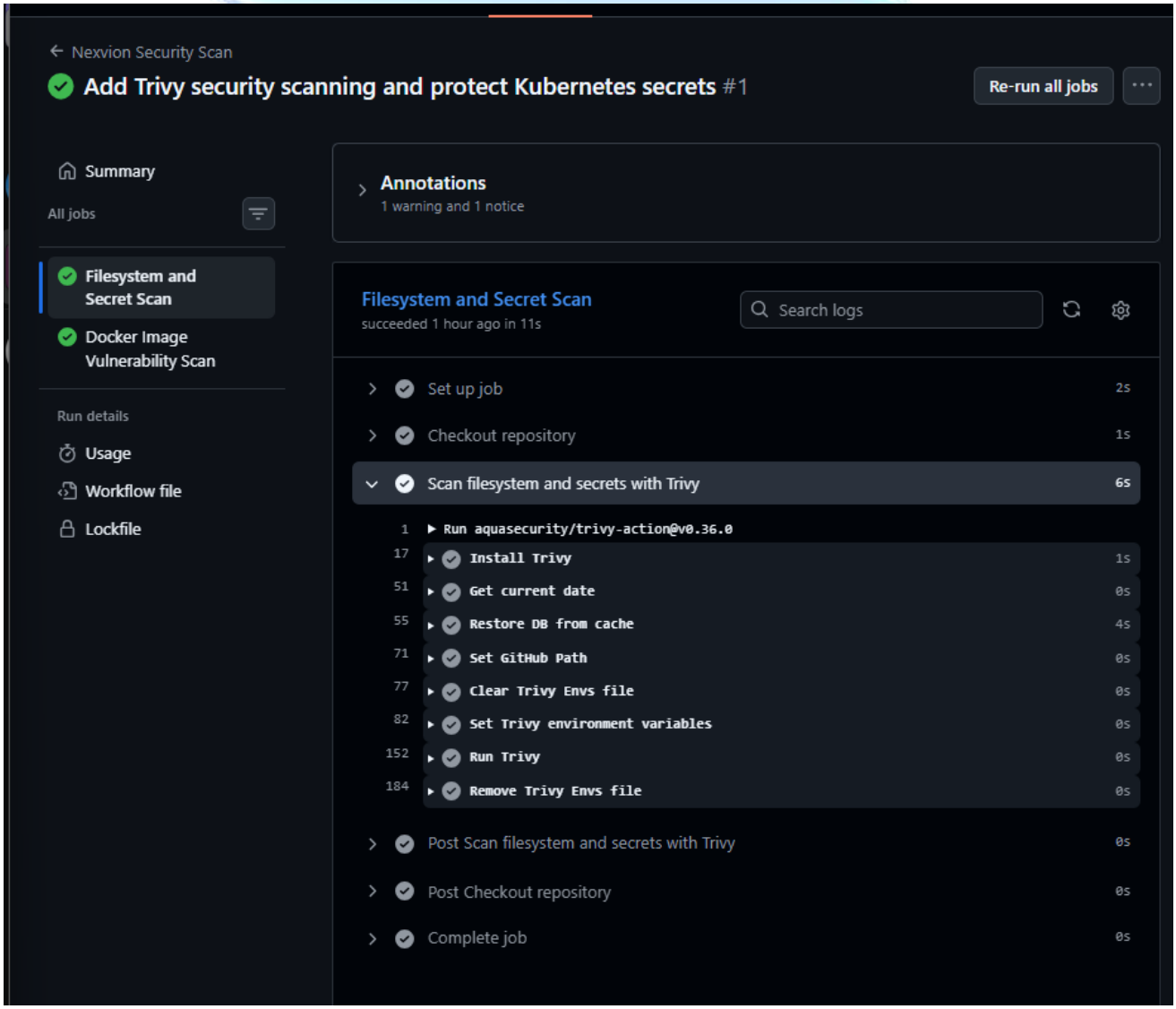


Figure 12.16: *Trivy filesystem and secret scan.*

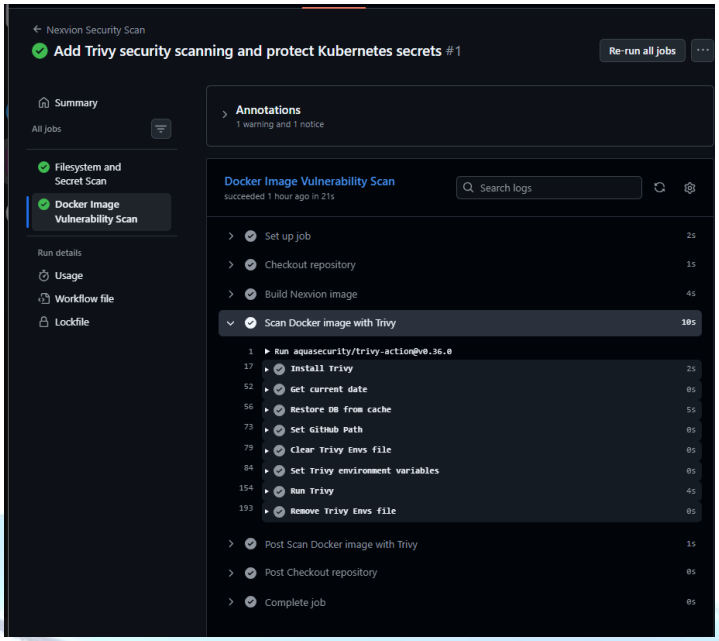


Figure 12.17: *Trivy Docker image scan.*

12.2 Final Output Evidence

The final output evidence should demonstrate that the application and its DevOps implementation were successfully tested.

Include:

1. Nexvion application homepage.
2. Nexvion products page.
3. Nexvion payment demonstration page.
4. Candidate application health check.
5. Production application health check.
6. Kubernetes application output.
7. Grafana monitoring dashboard.
8. Successful CI/CD workflow.
9. Successful security workflow.
10. GitHub repository showing the final project files and README.

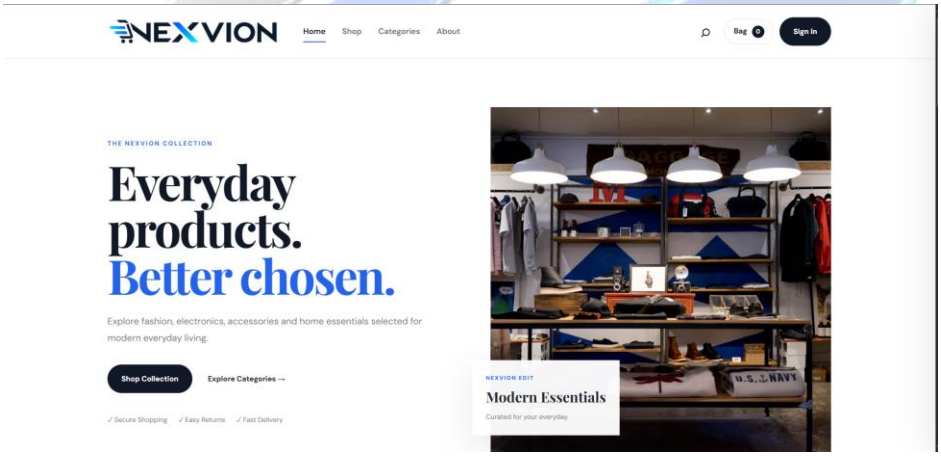


Figure 12.18: *Final Nexvion application output (homepage, products page, payment demonstration page).*

13. Challenges & Solutions

13.1 Challenges Faced

During the implementation of Nexvion, several practical challenges were encountered. These challenges helped in understanding real-world DevOps troubleshooting and system administration.

1. Docker Container and Deployment Management

Challenge: Managing multiple application containers and distinguishing between candidate and production deployments required careful port and container management.

Solution: Separate deployment ports were used for candidate and production environments. Container status and application health were checked before proceeding with deployment.

2. Self-hosted GitHub Actions Runner

Challenge: Deployment jobs depended on the availability of the self-hosted runner. The runner needed to be active on the Ubuntu server for deployment operations.

Solution: The runner was configured on the server and executed using its `run.sh` script. Its availability was checked before running deployment workflows.

3. Terraform Infrastructure Configuration

Challenge: Infrastructure configuration required correct provider settings, variables, resource definitions, and state management.

Solution: Terraform configuration was organized using provider settings, input variables, resource definitions, and outputs. State and plan files were kept out of Git tracking to avoid committing local infrastructure state.

4. Ansible Python Compatibility

Challenge: The Ansible GitHub Actions workflow encountered a Python-version compatibility issue.

Solution: Python 3.12 was pinned in the workflow environment. The workflow was executed again and passed successfully.

5. Ansible Idempotency

Challenge: It was necessary to verify that repeated playbook execution would not introduce unnecessary configuration changes.

Solution: The playbook was executed again after the initial configuration. The result was `ok=9 changed=0 failed=0 unreachable=0`, confirming that no additional changes were required.

6. Kubernetes Deployment Failure

Challenge: An invalid Docker image version was used to simulate a failed Kubernetes deployment.

Solution: The failure was observed, and the Deployment was rolled back to the previous working image. The application returned to a healthy state with two running replicas.

7. Monitoring Application Availability

Challenge: Monitoring needed to detect when the candidate application became unavailable rather than only displaying system metrics.

Solution: Blackbox Exporter and Prometheus alert rules were configured. The candidate container was stopped to simulate an outage. The website-down alert fired, and recovery was verified after restarting the application.

8. Trivy Scan Configuration

Challenge: Security scanning needed to be integrated into GitHub Actions with clearly defined scan types and severity thresholds.

Solution: A dedicated security workflow was created with filesystem, secret, and Docker image scanning. The workflow was configured to enforce the selected HIGH and CRITICAL severity criteria.

9. Server Disk Space Management

Challenge: The Ubuntu server had limited disk space. Docker images, volumes, and the Trivy vulnerability database consumed significant storage.

Solution: Docker storage was inspected to identify unused images and containers. Unnecessary resources were cleaned up carefully. The Trivy cache was also identified as a major storage consumer. Cleanup was approached cautiously to avoid removing resources required by running services or future security scans.

10. Maintaining a Clean Git Repository

Challenge: Temporary files, local configuration, and generated artifacts needed to be prevented from entering the Git repository.

Solution: `.gitignore` rules were used for local Terraform state, plan files, and sensitive or temporary configuration. Empty temporary files were removed, and the repository was checked to confirm a clean working tree.

13.2 Solutions Applied

The challenges were resolved through systematic troubleshooting and validation.

The following practices were applied:

- Checked container status and logs before changing deployments.
- Verified application availability through HTTP health checks.
- Ensured the self-hosted runner was active.
- Used Terraform variables and outputs to organize infrastructure configuration.
- Pinned compatible Python versions in automation workflows.
- Repeated Ansible execution to verify idempotency.
- Used Kubernetes rollback to restore a working deployment.
- Simulated an application outage to validate alerting.
- Used Trivy security scanning in automated workflows.
- Inspected disk usage before removing unnecessary resources.
- Maintained a clean Git working tree and excluded local state files.

These troubleshooting activities improved the reliability and maintainability of the project.

14. Learning Outcomes

14.1 Technical Learning

The project provided practical knowledge of the following technologies and concepts:

Docker and Containerization

Learned how to create Docker images, run containers, inspect logs, manage container lifecycle, and use Docker Compose.

CI/CD Automation

Learned how to create GitHub Actions workflows, execute automated checks, build Docker images, publish images, and deploy applications.

GitHub Container Registry

Learned how to publish and manage container images through GHCR.

Linux Administration

Improved knowledge of Ubuntu server management, command-line troubleshooting, service management, storage inspection, and application deployment.

Terraform

Learned the fundamentals of Infrastructure as Code, AWS provider configuration, security group resources, variables, outputs, and Terraform state management.

Ansible

Learned inventory management, playbook creation, package installation, Jinja2 templates, Nginx configuration, and idempotency.

Kubernetes

Learned how to create a local cluster, manage namespaces, Deployments, replicas, ConfigMaps, Services, and rollback operations.

Prometheus and Grafana

Learned how to collect monitoring metrics, configure targets, build dashboards, and create alert rules.

Trivy

Learned how to scan project files, detect potential secrets, scan Docker images, and configure security gates in CI/CD.

14.2 Practical / Industry Learning

Apart from technical knowledge, the project provided important practical experience.

1. **Troubleshooting:** Learned to identify problems using logs, status commands, and monitoring results.
2. **Automation:** Understood how repetitive deployment and configuration activities can be automated.
3. **Deployment Safety:** Learned the importance of health checks and rollback before considering a deployment successful.
4. **Infrastructure Management:** Understood how infrastructure can be represented and maintained through code.
5. **Monitoring:** Learned why continuous monitoring is important for application reliability.
6. **Security Awareness:** Understood the importance of integrating security checks into the development lifecycle.
7. **Version Control:** Gained experience maintaining project files and documentation using Git and GitHub.
8. **Documentation:** Learned to record implementation steps, test results, challenges, and evidence.
9. **Problem Solving:** Improved the ability to troubleshoot errors rather than relying only on predefined instructions.
10. **DevOps Lifecycle:** Understood how development, automation, deployment, monitoring, and security are connected.

15. Conclusion

15.1 Project Conclusion

The Nexvion project successfully demonstrated the practical implementation of an end-to-end DevOps workflow using multiple industry-relevant tools and technologies.

The project began with containerizing the Nexvion frontend application using Docker and progressed through CI/CD automation, infrastructure configuration, configuration management, Kubernetes orchestration, monitoring, and security scanning.

GitHub Actions was used to automate application checks, Docker image building, and publishing. A self-hosted runner supported deployment operations, while candidate and production environments were validated through health checks and rollback procedures.

Terraform demonstrated Infrastructure as Code through AWS security group configuration. Ansible automated Ubuntu server configuration and Nginx setup. Kubernetes was used to demonstrate application deployment, scaling through replicas, service exposure, rolling restart, and rollback in a local Kind cluster.

Prometheus and Grafana provided monitoring capabilities, while Blackbox Exporter and Prometheus alert rules helped validate website failure detection and recovery. Trivy was integrated into GitHub Actions to perform filesystem, secret, and Docker image scanning.

The project also provided practical experience in handling real implementation challenges such as Python compatibility, Kubernetes deployment failure, monitoring application availability, and limited server storage.

Overall, the project strengthened the understanding of the DevOps lifecycle and demonstrated how automation, infrastructure management, monitoring, and security practices can be combined to support reliable application delivery.

16. Future Enhancements

16.1 Possible Improvements

The Nexvion project can be extended in the following ways:

1. **Production Backend:** Develop a backend API and database to support real application data.
2. **Real Authentication:** Implement secure authentication, authorization, and session management.
3. **Payment Gateway:** Integrate a secure payment gateway for actual transaction processing.
4. **Cloud Deployment:** Extend the infrastructure implementation to deploy the complete application on AWS.
5. **Managed Kubernetes:** Deploy the application using a managed Kubernetes service such as Amazon EKS.
6. **Terraform Expansion:** Extend Terraform configuration to include VPC, subnets, compute resources, IAM roles, and other infrastructure components.
7. **Advanced CI/CD:** Add approval gates, deployment strategies, and more comprehensive automated testing.
8. **Centralized Logging:** Integrate a centralized logging solution for easier troubleshooting.
9. **External Alert Notifications:** Configure email or messaging notifications through Alertmanager integrations.
10. **Enhanced Security:** Add additional security checks, dependency updates, image signing, and secret management.
11. **High Availability:** Introduce multiple application instances and load balancing.
12. **Performance Testing:** Add load testing to understand application behavior under higher traffic.

These improvements would help move the demonstration project toward a more complete production-ready application.



17. References

17.1 Resources Used

The following official documentation and resources were used as references during implementation.

1. **Docker Documentation** – https://docs.docker.com/
2. **Docker Compose Documentation** – https://docs.docker.com/compose/
3. **Git Documentation** – https://git-scm.com/doc
4. **GitHub Actions Documentation** – https://docs.github.com/en/actions
5. **GitHub Container Registry Documentation** – https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry
6. **Terraform Documentation** – https://developer.hashicorp.com/terraform/docs
7. **AWS Documentation** – https://docs.aws.amazon.com/
8. **Ansible Documentation** – https://docs.ansible.com/
9. **Nginx Documentation** – https://nginx.org/en/docs/
10. **Kubernetes Documentation** – https://kubernetes.io/docs/
11. **Kind Documentation** – https://kind.sigs.k8s.io/
12. **Prometheus Documentation** – https://prometheus.io/docs/
13. **Grafana Documentation** – https://grafana.com/docs/
14. **Node Exporter Documentation** – https://github.com/prometheus/node_exporter
15. **Blackbox Exporter Documentation** – https://github.com/prometheus/blackbox_exporter
16. **Alertmanager Documentation** – https://prometheus.io/docs/alerting/latest/alertmanager/
17. **Trivy Documentation** – https://trivy.dev/latest/

Project Repository: https://github.com/CloudWithC/Nexvion